

LLM Watermarking

CS60216: Safety Fundamentals for Generative AI

Why to detect AI-generated text?

- Social engineering and election manipulation campaigns that exploit automated bots on social media platforms,
- Creation of fake news and web content, and
- Use of AI systems for cheating on academic writing and coding assignments

Major conference catches illicit AI use – and rejects hundreds of papers

The papers' watermarks allowed organizers to detect use of large language models in peer review.

By [Elizabeth Gibney](#)



Organizers of the 2026 International Conference on Machine Learning (ICML) used a watermarking system to catch the use of AI in peer review of conference papers. Credit: Stephen Barnes/Alamy

Why to detect AI-generated text?

Article | [Open access](#) | Published: 24 July 2024

AI models collapse when trained on recursively generated data

[Ilia Shumailov](#) , [Zakhar Shumaylov](#) , [Yiren Zhao](#), [Nicolas Papernot](#), [Ross Anderson](#) & [Yarin Gal](#) 

[Nature](#) **631**, 755–759 (2024) | [Cite this article](#)

691k Accesses | **567** Citations | **3717** Altmetric | [Metrics](#)

Can you distinguish human vs. machine generated text?

Through the town, and past the lights,
Oh, how the bells do ring!
They chime with glee
For you and me
As carols we joyfully sing.

Over the river, and through the wood,
Oh, how the wind does blow!
It stings the toes
And bites the nose
As over the ground we go.

Can you distinguish human vs. machine generated text?

Through the town, and past the lights,
Oh, how the bells do ring!
They chime with glee
For you and me
As carols we joyfully sing.



Machine

Over the river, and through the wood,
Oh, how the wind does blow!
It stings the toes
And bites the nose
As over the ground we go.

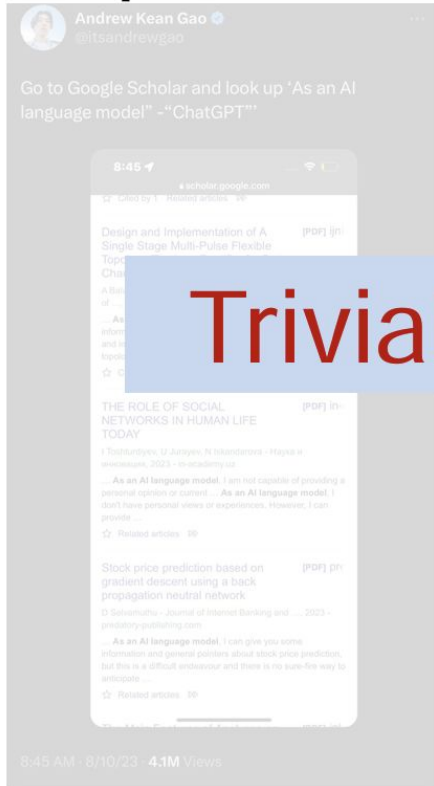


Human

Child, Lydia Maria. "Thanksgiving Day." 1844.

How to detect AI-generated text?

- Add prefix: “As a large language model...”



III. PROPOSED SYSTEM

As an AI language model, I can provide some general information on the proposed system for the analysis, design, and implementation of a single-stage multi-pulse flexible-topology thyristor rectifier for battery charging in electric vehicles. The proposed system aims to develop a high-efficiency and reliable battery charging system for electric vehicles. The system utilizes a single-stage multi-pulse flexible-topology thyristor rectifier to charge the battery. This rectifier has a flexible topology that is suitable for charging electric vehicles. It has a control mechanism that ensures the battery is charged in a safe and healthy manner. The control mechanism regulates the rectifier's output voltage and current to ensure that the battery is charged in a controlled and safe manner. It also monitors the battery's status and adjusts the charging process accordingly to prevent overcharging or overheating. The proposed system is designed to be compact and lightweight, making it easy to install and use in electric vehicles. It is also designed to be cost-effective while providing high performance and reliability, which is essential for the widespread adoption of electric vehicles. The proposed system for the analysis, design, and implementation of a single-stage multi-pulse flexible-topology thyristor rectifier for battery charging in electric vehicles is a promising solution for the development of efficient and reliable battery charging systems for electric vehicles.

Trivial to remove from text!

How to detect AI-generated text?

- Train classification models [GPTZero, Turnitin, ...]

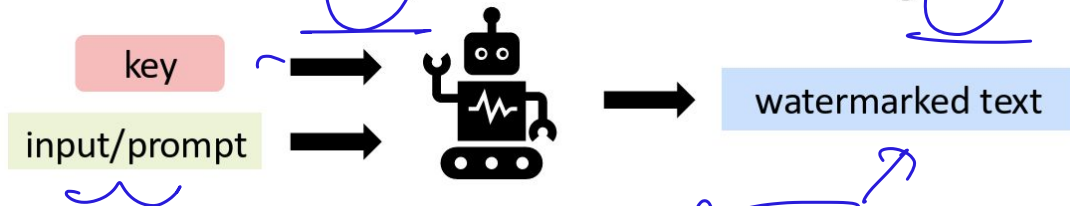
Too many false positives?
Out-of-distribution data?
AI development?

Why watermarking is promising?

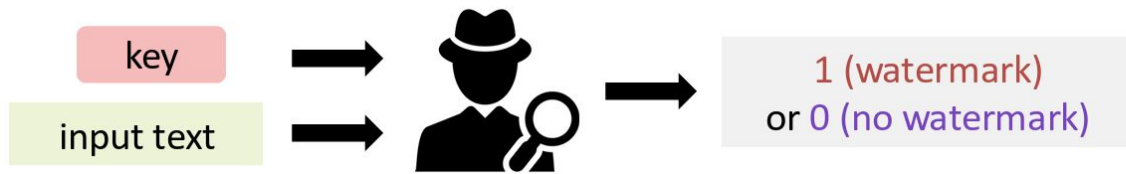
Plant subtle but distinctive signals deliberately within the content to enable downstream detection

An LM Watermarking Scheme has two components

- **Watermark($\mathcal{M}$)**: (possibly randomized procedure) that outputs a new model $\hat{\mathcal{M}}$, and detection key k



- **Detect($k, \mathbf{y}$)**: takes input detection key k and sequence $\mathbf{y}$, then outputs 1 (indicating it was AI-generated) or 0 (indicating it was human-generated)



Four Key Metrics of a Watermarking Scheme



Quality

Watermark should not degrade the quality of the model in any way.



Detectability

Ability to plant subtle but distinctive signals for downstream detection. *More on this in the next slide.*



Robustness

Be robust against attacks, such as post-editing or other adversarial modifications.

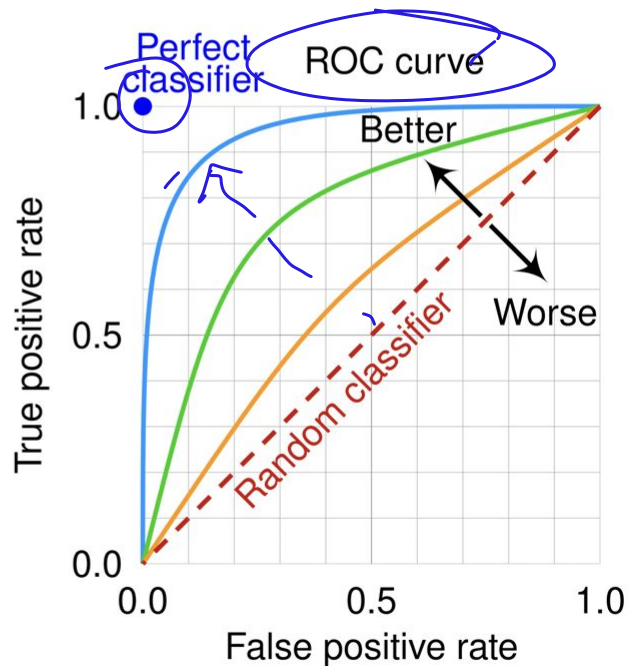


Unlearnability (Security)

Cannot easily create the watermarked content without the key.

Detectability: A hypothesis testing view of LLM watermarks

- H_0 : The suspect text y is NOT generated from $\hat{\mathcal{M}}$
e.g., “ y ” is written by a human.
e.g., “ y ” is generated by $\mathcal{M}$.
- H_1 : The suspect text is generated from $\hat{\mathcal{M}}$
A very broad “Null” and a very specific “Alternative”
- **Theory** : Can we control FPR. Can we prove high power? Are the tradeoff optimal?



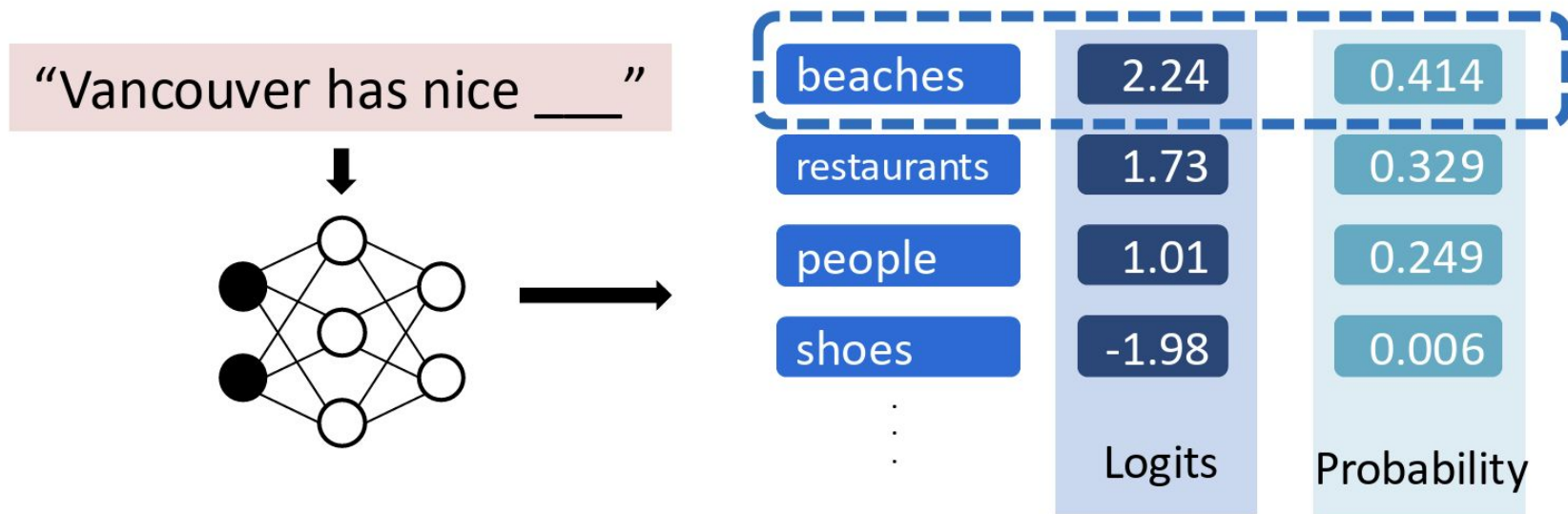
Green-Red Watermarks

Outputs of a language model, both with and without the application of a watermark. The watermarked text, if written by a human, is expected to contain 9 “green” tokens, yet it contains 28. The probability of this happening by random chance is $\approx 6 \times 10^{-14}$, leaving us extremely certain that this text is machine generated.

Prompt	Num tokens	Z-score	p-value
...The watermark detection algorithm can be made public, enabling third parties (e.g., social media platforms) to run it themselves, or it can be kept private and run behind an API. We seek a watermark with the following properties:			
No watermark Extremely efficient on average term lengths and word frequencies on synthetic, microamount text (as little as 25 words) Very small and low-resource key/hash (e.g., 140 bits per key is sufficient for 99.999999999% of the Synthetic Internet	56	.31	.38
With watermark ✓ ? - minimal marginal probability for a detection attempt. - Good speech frequency and energy rate reduction. - messages indiscernible to humans. - easy for humans to verify.	36	7.4	6e-14

What is a language model?

$$P(\text{next word } y_t \mid \text{Prompt } x, \text{previous words } y_{1:t-1})$$



The **universe of words** is called a **vocabulary** V

Hard Red List

A third party with knowledge of the ***hash function*** and ***random number generator*** can re-produce the red list for each token and count how many times the red list rule is violated.

Algorithm 1 Text Generation with Hard Red List

Input: prompt, $s^{(-N_p)} \dots s^{(-1)}$

for $t = 0, 1, \dots$ **do**

1. Apply the language model to prior tokens $s^{(-N_p)} \dots s^{(t-1)}$ to get a probability vector $p^{(t)}$ over the vocabulary.
2. Compute a hash of token $s^{(t-1)}$, and use it to seed a random number generator.
3. Using this seed, randomly partition the vocabulary into a “green list” G and a “red list” R of equal size.
4. Sample $s^{(t)}$ from G , never generating any token in the red list.

end for

Full Worked Example

Say vocabulary = {cat, dog, sat, mat, the, a}, and the previous token was "the" (token ID = 4).

seed = hash(4) = 2847392810

RNG(2847392810).shuffle(vocab)
→ [dog, the, mat, cat, sat, a]

Green G = {dog, the, mat}

Red R = {cat, sat, a}

LM probabilities:

cat → 0.30 ✗ red, zeroed

dog → 0.25 ✓ green

sat → 0.20 ✗ red, zeroed

mat → 0.15 ✓ green

the → 0.07 ✓ green

a → 0.03 ✗ red, zeroed

After renormalization (sum of green = 0.47):

dog → 0.532 ✓

mat → 0.319 ✓

the → 0.149 ✓

Sample → "dog" (most likely)

Hard Red List

H_0 : The text sequence is generated with no knowledge of the red list rule.

Because the red list is chosen at random, a natural writer is expected to violate the red list rule with **half of their tokens**, while the watermarked model produces no violations.

The probability that a natural source produces T tokens without violating the red list rule is only $1/2^T$, which is vanishingly small even for short text fragments with a dozen words. This enables detection of the watermark (rejection of H_0) for, e.g., a synthetic tweet.

Hard Red List: one proportion z-test

If the null hypothesis is true, then the number of green list tokens, denoted $|s|_G$ has expected value $T/2$ and variance $T/4$. (Why? Bernoulli Distribution)

The z-statistic for this test is

seq. length = T

$$z = 2(|s|_G - T/2) / \sqrt{T} \quad \leftarrow \quad \frac{|s|_G - T/2}{\sqrt{T/4}}$$

We reject the null hypothesis and detect the watermark if z is above a chosen threshold.

Hard Red List: one proportion z-test

$$z = 2(|s|_G - T/2)/\sqrt{T}$$

$$\frac{(11 + 48 - 4)}{2} = 4$$

Suppose we choose to reject the null hypothesis if $z > 4$.

In this case, the probability of a false positive is 3×10^{-5} , which is the one-sided p-value corresponding to $z > 4$.

Can you detect watermarking for any length of text?

$$4 = \frac{2(T - T/2)}{\sqrt{T}} \Rightarrow 2 = \frac{T}{2\sqrt{T}} \Rightarrow \sqrt{T} = 4 \Rightarrow \underline{T = 16}$$

Hard Red List: one proportion z-test

$$z = 2(|s|_G - T/2)/\sqrt{T}$$

Suppose we choose to reject the null hypothesis if $z > 4$.

In this case, the probability of a false positive is 3×10^{-5} , which is the one-sided p-value corresponding to $z > 4$.

[We will detect any ~~watermarked~~ sequence with 16 or more tokens] (the minimum value of T that produces $z = 4$ when $|s|_G = T$).

$|S_G| = 1000$ initially

after attack
 $|S_G|_{adv} = \cancel{800} \underline{600}$

$$z = \frac{2(600 - 500)}{\sqrt{2}} =$$

What if an adversary modifies some tokens?

Consider the case of a watermarked sequence of length $T = 1000$.

Suppose an adversary modifies 200 tokens in the sequence to (potentially) add red list words and scrub the watermark.

A modified token at position t can violate the red list rule at position t .

Furthermore, the value of s_t determines the red list for token s_{t+1} , and a maximally adversarial choice of s_t will put s_{t+1} in violation of the red list rule as well.

For this reason, 200 token flips can create at most 400 violations of the red list rule.

What if an adversary modifies some tokens?

Consider the case of a watermarked sequence of length $T = 1000$.

Suppose an adversary modifies 200 tokens in the sequence to (potentially) add red list words and scrub the watermark.

200 token flips can create at most 400 violations of the red list rule.

This maximally adversarial sequence with 600 remaining green list tokens still produces a z-statistic of $2(600 - 1000/2)/\sqrt{1000} \approx 6.3$, and a p-value of $\approx 10^{-10}$,

$$z = \frac{2(x - 500)}{\sqrt{1000}} \quad x \approx 564 \quad \sqrt{220}$$

A soft watermark: **Case of low entropy tokens**

Consider the following two sequences of tokens, with prompts in red:

The quick brown fox jumps over the lazy dog

for(i=0;i<n;i++) sum+=array[i]

Were they produced by a human or by a language model?

A soft watermark: Case of low entropy tokens

Consider the following two sequences of tokens, with prompts in red:

The quick brown fox jumps over the lazy dog

```
for(i=0;i<n;i++) sum+=array[i]
```

Were they produced by a human or by a language model?

- Determining this is fundamentally hard because these sequences have low entropy
- It is difficult to watermark low entropy text, as any changes to the choice of tokens may result in high perplexity, unexpected tokens

Soft watermark: A more sophisticated framework

We now discuss the “soft” watermark that promotes the

- Use of the green list for high entropy tokens when many good choices are available, while
- Having little impact on the choice of low-entropy tokens that are nearly deterministic.

How?

Soft watermark: A more sophisticated framework

We now discuss the “soft” watermark that promotes the

- Use of the green list for high entropy tokens when many good choices are available, while
- Having little impact on the choice of low-entropy tokens that are nearly deterministic.

How?

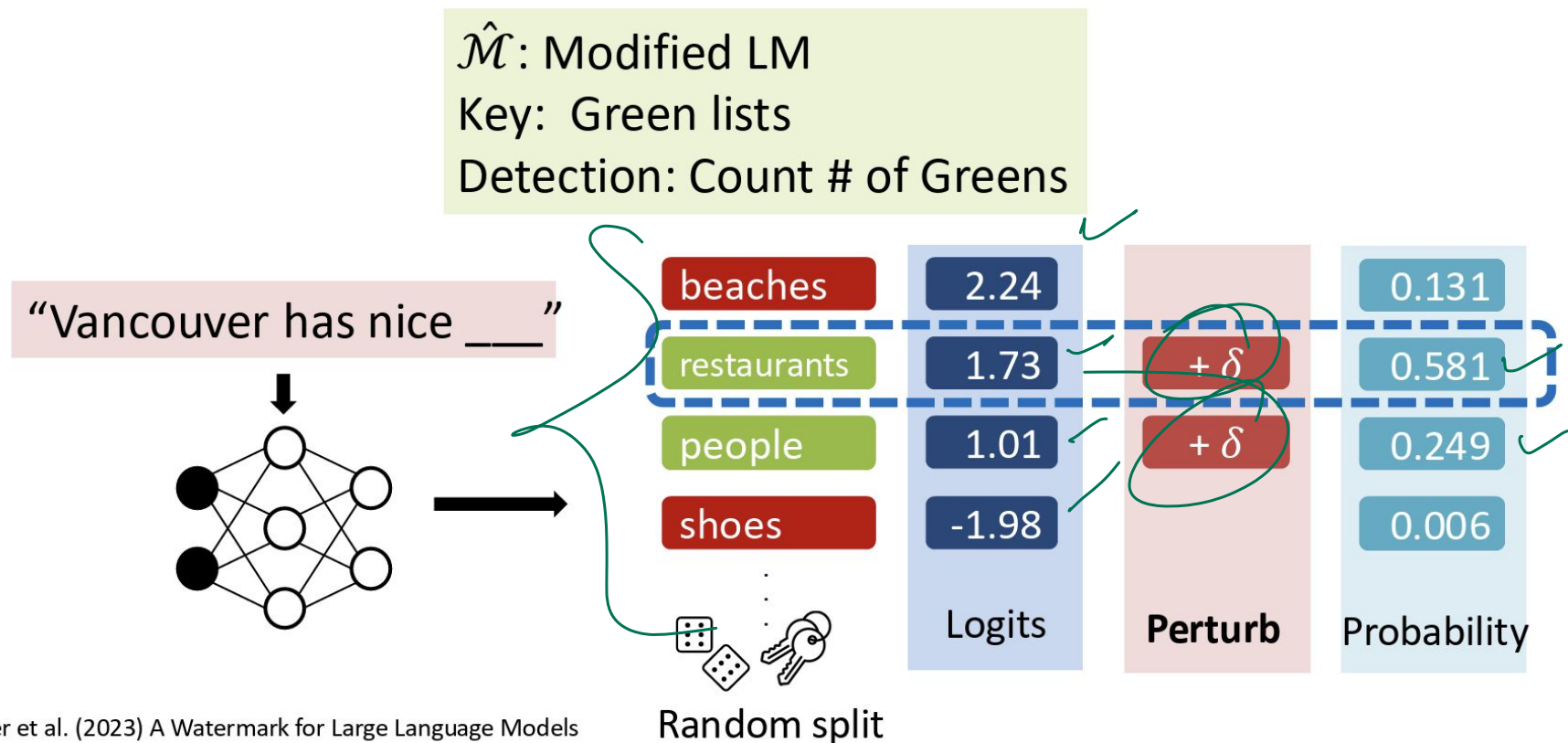
The last layer of the language model outputs a vector of logits $l^{(t)}$. These logits get converted into a probability vector $p^{(t)}$ using the softmax operator

$$p_k^{(t)} = \exp(l_k^{(t)}) / \sum_i \exp(l_i^{(t)})$$

Basic Idea: add a constant δ to the logits of the green list tokens.

Green-Red Watermark

(Kirchenbauer et al. 2023; Zhao et al. 2023)



Green-Red Watermark

(Kirchenbauer et al. 2023; Zhao et al. 2023)

$$\mathcal{M}: y_t \sim \text{Softmax}(\text{logits}(\text{Prompt}, y_{<t}))$$

$$\hat{\mathcal{M}}: y_t \sim \text{Softmax}(\underbrace{\text{logits}(\text{Prompt}, y_{<t})}_{\text{original logits}} + \underbrace{\delta \cdot \mathbf{1}(\cdot \text{ is green})}_{\text{green bias}})$$

Increase the probability of green tokens slightly.

Decrease the probability of red tokens slightly.

Soft Red List

A highly likely word with $p_k^{(t)} \approx 1$ has a much larger logit than other candidates, and this will remain the largest regardless of whether it is in the red list.

But when the entropy is high, there are many comparably large logits to choose from, and the δ rule has a large impact on the sampling distribution, strongly biasing the output towards the green list.

Algorithm 2 Text Generation with Soft Red List

Input: prompt, $s^{(-N_p)} \dots s^{(-1)}$
green list size, $\gamma \in (0, 1)$
hardness parameter, $\delta > 0$

for $t = 0, 1, \dots$ **do**

1. Apply the language model to prior tokens $s^{(-N_p)} \dots s^{(t-1)}$ to get a logit vector $l^{(t)}$ over the vocabulary.
2. Compute a hash of token $s^{(t-1)}$, and use it to seed a random number generator.

Using this random number generator, randomly partition the vocabulary into a “green list” G of size $\gamma|V|$ and a “red list” R of size $(1 - \gamma)|V|$.

4. Add δ to each green list logit. Apply the softmax operator to these modified logits to get a probability distribution over the vocabulary.

$$\hat{p}_k^{(t)} = \begin{cases} \frac{\exp(l_k^{(t)} + \delta)}{\sum_{i \in R} \exp(l_i^{(t)}) + \sum_{i \in G} \exp(l_i^{(t)} + \delta)}, & k \in G \\ \frac{\exp(l_k^{(t)})}{\sum_{i \in R} \exp(l_i^{(t)}) + \sum_{i \in G} \exp(l_i^{(t)} + \delta)}, & k \in R. \end{cases}$$

5. Sample the next token, $s^{(t)}$, using the water-marked distribution $\hat{p}^{(t)}$.

end for

Detecting the soft watermark

For arbitrary γ ($=1/2$ previously) we have

$$\gamma = 1/2$$

$$z = (|s|_G - \gamma T) / \sqrt{T\gamma(1-\gamma)}$$

Why? Bernoulli Distribution

$$\frac{(|s|_G - T/2)}{\sqrt{\frac{T}{2} \cdot \frac{1}{2}}}$$

Detection of Green-Red Watermark

Input: Suspect text $y = [y_1, \dots, y_n]$, e.g. "Over the ..."

(Optional pre-processing) $y = \text{unique}(y)$

1. Compute the **z-score**:

$$z = (|y|_G - \gamma n) / \sqrt{n\gamma(1 - \gamma)}$$

2. If $z > \text{threshold}$ then

Return "y is watermarked"

Else

Return "no evidence"

Num of **Green tokens**

Green-Red Watermark Examples

Prompt: Can I succeed after many failures?

LLaMA-13B, unwatermarked
z-score = -2.4

A: Of course it is, and that is how we improve. Saying "I can't do that" is never a good thing. Sometimes we think we've tried all we can and that "isn't enough". That is the time when we ask for help. The root of all evils is to be a secret. Honesty and self-criticism is necessary for improvement. The measure of intelligence is the ability to change. [continues...]

Prompt: Can I succeed after many failures?

LLaMA-13B, watermarked
z-score = 11

A: When most people are confronted with failure, they cannot imagine such a thing happening. When one faces business reverses and bankruptcy, it seems impossible. When we are rejected it looks as if we are going to be rejected forever. However, it does not need to be this way. The human spirit simply will not give up. [continues...]

Different versions of Green-Red WM

- Green-Red watermark for code generation (Lee et al., 2023; Guan et al., 2024)
- Adaptive/dynamic perturbations in the logits (Liu et al., 2023; Huo et al., 2024, Liu et al., 2024)
- Public key (Liu et al., 2023; Zhou et al., 2024)
- Multi-bits (Yoo et al., 2023; Fernandez et al., 2023)
- Many others...

Reference



https://leililab.github.io/llm_watermark_tutorial/

<https://openreview.net/pdf?id=aX8ig9X2a7> *[Analysis of soft watermarking]*

